

□ Les Méthodes Magiques en Python

Le support est une propriété de **Bechir Bejaoui**

1 □ Pourquoi les méthodes magiques ?

En Python, quand vous écrivez `len(ma_liste)` ou `obj1 + obj2`, Python ne fait pas de magie —il appelle des méthodes bien précises sur vos objets. Ces méthodes s'appellent les **dunder methods** (*double underscore*) ou **méthodes magiques**.

Le problème sans méthodes magiques :

```
class Temperature:
    def __init__(self, valeur):
        self.valeur = valeur

t1 = Temperature(100)
t2 = Temperature(50)

# Tout cela échoue sans méthodes magiques :
print(t1)          # <__main__.Temperature object at 0x...> ← illisible
print(t1 + t2)      # TypeError: unsupported operand type(s) for +
print(t1 > t2)      # TypeError: '>' not supported
print(len(t1))      # TypeError: object of type 'Temperature' has no len()
```

- `<__main__.Temperature object at 0x...>` : c'est ce que Python affiche par défaut —totalement inutile
- **TypeError** : Python ne sait pas quoi faire avec `+` ou `>` sur un objet inconnu
- **conclusion** : la classe existe, mais elle est "muette"—elle ne parle pas la langue de Python

Avec les méthodes magiques :

```
class Temperature:
    def __init__(self, valeur: float, unite: str = "°C"):
        self.valeur = valeur
        self.unite = unite

    def __str__(self):
        # print(obj)
        return f"{self.valeur}{self.unite}"

    def __repr__(self):
        # repr(obj), Logs
        return f"Temperature({self.valeur!r}, {self.unite!r})"
```

```

def __add__(self, other):                # t1 + t2
    return Temperature(self.valeur + other.valeur, self.unite)

def __gt__(self, other):                 # t1 > t2
    return self.valeur > other.valeur

def __eq__(self, other):                 # t1 == t2
    return self.valeur == other.valeur

def __bool__(self):                     # if t:
    return self.valeur > 0

t1 = Temperature(100)
t2 = Temperature(50)

print(t1)                               # 100°C           ← __str__
print(repr(t1))                          # Temperature(100, '°C') ← __repr__
print(t1 + t2)                           # 150°C           ← __add__
print(t1 > t2)                            # True            ← __gt__
print(bool(t2))                          # True            ← __bool__

```

- **__str__** : ce que `print()` affiche —pensez “message pour l'utilisateur final”
- **__repr__** : représentation technique —pensez “message pour le développeur”
- **__add__** : intercepte l'opérateur `+` —retourne **toujours** un objet du même type
- **__gt__** : intercepte `>` —Python déduit `<` depuis **__gt__** dans certains contextes, mais il vaut mieux tout déclarer
- **__bool__** : définit ce que vaut `if obj:` —par défaut tout objet est `True` sans cette méthode

Vue d'ensemble des familles de méthodes magiques :

Famille	Méthodes	Utilité
Cycle de vie	<code>__new__</code> , <code>__init__</code> , <code>__del__</code>	Création, init, destruction
Représentation	<code>__str__</code> , <code>__repr__</code> , <code>__format__</code>	Affichage, logs, f-strings
Opérateurs arithmétiques	<code>__add__</code> , <code>__sub__</code> , <code>__mul__</code> , <code>__truediv__</code>	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code>
Opérateurs de comparaison	<code>__eq__</code> , <code>__lt__</code> , <code>__le__</code> , <code>__gt__</code> , <code>__ge__</code>	<code>==</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>

Famille	Méthodes	Utilité
Conteneurs	<code>__len__</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>__contains__</code> , <code>__iter__</code>	<code>len()</code> , <code>[]</code> , <code>in</code> , <code>for</code>
Attributs	<code>__getattr__</code> , <code>__setattr__</code> , <code>__getattribute__</code>	Interception des accès
Callable & contexte	<code>__call__</code> , <code>__enter__</code> , <code>__exit__</code>	<code>obj()</code> , <code>with obj</code>
Hachage & vérité	<code>__hash__</code> , <code>__bool__</code>	<code>dict</code> , <code>set</code> , <code>if obj</code>

2 □ Comment Python décide quoi appeler ?

Chaque expression Python est en réalité un **appel de méthode magique déguisé**. Comprendre ce mécanisme, c'est comprendre le Data Model Python.

Ce que vous écrivez → Ce que Python appelle réellement

<code>print(obj)</code>	→ <code>obj.__str__()</code>
<code>len(obj)</code>	→ <code>obj.__len__()</code>
<code>obj1 + obj2</code>	→ <code>obj1.__add__(obj2)</code>
<code>obj1 == obj2</code>	→ <code>obj1.__eq__(obj2)</code>
<code>obj[2]</code>	→ <code>obj.__getitem__(2)</code>
<code>obj[2] = "x"</code>	→ <code>obj.__setitem__(2, "x")</code>
<code>2 in obj</code>	→ <code>obj.__contains__(2)</code>
<code>for x in obj</code>	→ <code>iter(obj)</code> → <code>obj.__iter__()</code>
<code>obj()</code>	→ <code>obj.__call__()</code>
<code>with obj as x</code>	→ <code>obj.__enter__()</code> / <code>obj.__exit__(...)</code>
<code>bool(obj)</code>	→ <code>obj.__bool__()</code>
<code>abs(obj)</code>	→ <code>obj.__abs__()</code>

Démonstration —espionner les appels Python :

class Espion:

"""Classe qui affiche chaque méthode magique appelée par Python."""

```
def __init__(self, nom):
    # On utilise object.__setattr__ pour éviter d'appeler notre propre __setattr__
    object.__setattr__(self, "nom", nom)

def __str__(self):
    print(f" → __str__ appelé")
    return f"Espion({self.nom})"
```

```

def __len__(self):
    print(f" → __len__ appelé")
    return 42

def __add__(self, other):
    print(f" → __add__ appelé avec {other}")
    return Espion(f"{self.nom}+{other.nom}")

def __contains__(self, item):
    print(f" → __contains__ appelé avec {item!r}")
    return item == "x"

def __bool__(self):
    print(f" → __bool__ appelé")
    return True

e1 = Espion("A")
e2 = Espion("B")

print("\n--- print(e1) ---")
print(e1)

print("\n--- len(e1) ---")
print(len(e1))

print("\n--- e1 + e2 ---")
e3 = e1 + e2
print(e3)

print("\n--- 'x' in e1 ---")
print("x" in e1)

print("\n--- if e1 ---")
if e1:
    print(" e1 est truthy")

--- print(e1) ---
→ __str__ appelé
Espion(A)

--- len(e1) ---
→ __len__ appelé
42

--- e1 + e2 ---

```

→ `__add__` appelé avec `Espion(B)`
 → `__str__` appelé
`Espion(A+B)`

--- 'x' in e1 ---
 → `__contains__` appelé avec 'x'
`True`

--- if e1 ---
 → `__bool__` appelé
`e1` est truthy

- **chaque opérateur** est une façade —Python cherche la méthode correspondante et l'appelle
- **object.__setattr__** : permet d'écrire un attribut en contournant notre propre `__setattr__` —indispensable pour éviter la récursion infinie
- **si la méthode est absente** : Python lève `TypeError` avec un message explicite —jamais de comportement silencieux

Le mécanisme de fallback —opérateurs réfléchis :

```
class Euro:
    def __init__(self, valeur):
        self.valeur = valeur

    def __add__(self, other):
        print(f" Euro.__add__ appelé")
        if isinstance(other, (int, float)):
            return Euro(self.valeur + other)
        return NotImplemented    # ← signal à Python d'essayer l'opération inverse

    # Si l'opération échoue aussi, python lève erreur
    def __radd__(self, other):
        print(f" Euro.__radd__ appelé (opération inversée)")
        return Euro(self.valeur + other)

    def __str__(self):
        return f"{self.valeur}€"

e = Euro(100)

print(e + 50)    # Euro.__add__ appelé → 150€
print(50 + e)    # int.__add__ échoue → Python essaie Euro.__radd__ → 150€
```

- **NotImplemented** (pas une exception) : valeur spéciale que `__add__` retourne pour dire à Python "je ne sais pas gérer ça, essaie l'autre sens"

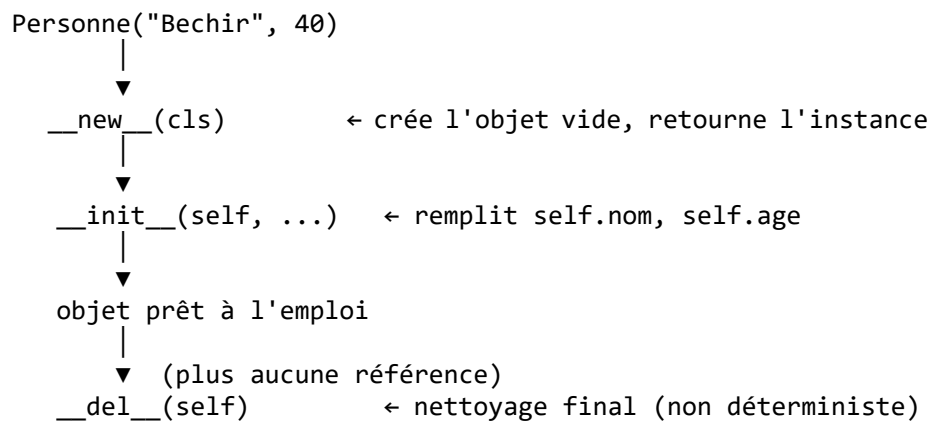
- `__radd__` : version *réfléchie* de `__add__` —appelée quand `other + self` échoue côté `other`
- Chaque opérateur a sa version réfléchie : `__radd__`, `__rsub__`, `__rmul__`
- ...

3 □ Cycle de vie d'un objet : `__new__` vs `__init__` vs `__del__`

Concept

La création d'un objet en Python passe par **deux étapes distinctes** que la plupart des développeurs confondent :

- `__new__` → **fabrique** l'objet vide (alloue la mémoire) —doit retourner l'instance
- `__init__` → **configure** l'objet déjà fabriqué (remplit les attributs) —ne retourne rien
- `__del__` → appelé par le **garbage collector** juste avant la destruction —moment non garanti



Observer les étapes de creation d'un objet

```
class Personne:
```

```

def __new__(cls, nom: str, age: int):
    print(f" 1. __new__ : allocation mémoire pour une Personne")
    instance = super().__new__(cls)  # ← alloue l'objet Python brut
  
```

```

# À ce stade : instance existe, mais self.nom et self.age n'existent PAS encore
print(f"    instance créée : {type(instance)}, id={id(instance)}")
return instance                                # OBLIGATOIRE — sinon __init__ n'est pas appelé

def __init__(self, nom: str, age: int):
    print(f"    2. __init__ : initialisation de l'objet")
    self.nom = nom        # maintenant self existe, on peut créer les attributs
    self.age = age
    print(f"    self.nom={self.nom}, self.age={self.age}")

def __del__(self):
    # Attention : __del__ peut être appelé après la fermeture de l'interpréteur
    # → ne jamais dépendre de ressources externes ici
    print(f"    3. __del__ : '{self.nom}' supprimé de la mémoire")

def __str__(self):
    return f"Personne({self.nom}, {self.age} ans)"

print("=== Création ===")
p = Personne("Bechir", 40)
print(f"Objet prêt : {p}\n")

print("=== Suppression explicite ===")
del p    # force la suppression — __del__ appelé immédiatement ici
        # (car plus aucune référence)

=== Création ===
1. __new__ : allocation mémoire pour une Personne
   instance créée : <class '__main__.Personne'>, id=140234...
2. __init__ : initialisation de l'objet
   self.nom=Bechir, self.age=40
Objet prêt : Personne(Bechir, 40)

=== Suppression explicite ===
3. __del__ : 'Bechir' supprimé de la mémoire
• super().__new__(cls) : délègue l'allocation réelle à object — ne jamais allouer manuellement
• si __new__ ne retourne pas une instance de cls : Python n'appelle pas __init__ — bug silencieux garanti
• __del__ : ne pas confondre avec del obj — del retire une référence, __del__ est appelé quand le compteur de références atteint 0
• à éviter dans __del__ : tout ce qui dépend d'autres objets (ils peuvent déjà être détruits) — préférer les context managers (with) pour la gestion de ressources

```

Observer l'objet dans la mémoire

```
import tracemalloc

class Personne:

    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

# démarrer le suivi mémoire
tracemalloc.start()

# création d'objets
personnes = [Personne("User", i) for i in range(10000)]

# récupérer la mémoire utilisée
current, peak = tracemalloc.get_traced_memory()

print(f"mémoire actuelle : {current / 1024:.2f} KB")
print(f"pic mémoire      : {peak / 1024:.2f} KB")

# arrêter le suivi
tracemalloc.stop()
```

A quoi sert réellement tracemalloc?

Ce module permet de :

- suivre les allocations mémoire Python
 - comparer l'utilisation mémoire entre deux moments
 - détecter les fuites mémoire
-

Cas pratique d'emploi de new le Singleton

```
class ConfigApp:
    """Configuration globale — une seule instance possible."""
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            print(" Première instanciation — création unique")
            cls._instance = super().__new__(cls)
            cls._instance._initialisee = False # flag pour __init__
```

```

        return cls._instance    # toujours le même objet

    def __init__(self):
        if not self._initialisee:
            self.debug = False
            self.version = "1.0"
            self._initialisee = True
            print(f" Configuration initialisée")
            # sinon : __init__ est appelé mais on ignore la réinitialisation

cfg1 = ConfigApp()
cfg2 = ConfigApp()
cfg3 = ConfigApp()

print(f"\ncfg1 is cfg2 : {cfg1 is cfg2}")    # True
print(f"cfg1 is cfg3 : {cfg1 is cfg3}")    # True
cfg1.debug = True
print(f"cfg2.debug : {cfg2.debug}")        # True — même objet !

    Première instantiation — création unique
    Configuration initialisée
    (silencieux pour cfg2 et cfg3)

```

```

cfg1 is cfg2 : True
cfg1 is cfg3 : True
cfg2.debug : True

```

- **_instance** = **None** : attribut de classe —stocke la seule instance autorisée
- **_initialisee** : flag pour éviter que **__init__** ré-écrase la config à chaque **ConfigApp()** —piège classique du Singleton Python
- **use case** : connexion DB, fichier de config, logger global —tout ce qui doit exister en **un seul exemplaire**

4 □ **__str__** vs **__repr__** —représentation d'un objet

Concept

Deux méthodes, deux audiences, deux objectifs :

- **__str__** → pour l'**utilisateur final** —lisible, convivial, peut omettre des détails

- `__repr__` → pour le **développeur** —précis, complet, idéalement `eval(repr(obj)) == obj`

<code>print(obj)</code>	→ <code>__str__</code> (si absent → fallback sur <code>__repr__</code>)
<code>repr(obj)</code>	→ <code>__repr__</code>
<code>f"{obj}"</code>	→ <code>__str__</code>
<code>f"{obj!r}"</code>	→ <code>__repr__</code>
<code>[obj1, obj2]</code>	→ <code>__repr__</code> de chaque élément
logs / debugging	→ <code>__repr__</code>
shell interactif	→ <code>__repr__</code>

Exemple complet —les deux en action

```
from datetime import date

class Employe:
    def __init__(self, nom: str, poste: str, salaire: float, embauche: date):
        self.nom = nom
        self.poste = poste
        self.salaire = salaire
        self.embauche = embauche

    def __str__(self) -> str:
        # Version lisible pour l'utilisateur
        return f"{self.nom} — {self.poste} ({self.salaire} TND/mois)"

    def __repr__(self) -> str:
        # Version reproductible pour le développeur
        return (
            f"Employe("
            f"nom={self.nom!r}, "
            f"poste={self.poste!r}, "
            f"salaire={self.salaire!r}, "
            f"embauche={self.embauche!r})"
        )

    def __format__(self, format_spec: str) -> str:
        # Contrôle f"{obj:court}" vs f"{obj:complet}"
        if format_spec == "court":
            return self.nom
        elif format_spec == "complet":
            return f"{self.nom} ({self.poste}, depuis {self.embauche.year})"
        return self.__str__()
```

```
e = Employe("Bechir", "Architecte logiciel", 4500, date(2020, 3, 15))

print("--- Contextes d'affichage ---")
print(str(e))          # __str__ : Bechir — Architecte logiciel (4500 TND/mois)
print(repr(e))          # __repr__ : Employe(nom='Bechir', poste='Architecte...
print(f"{e}")           # __str__ : Bechir — Architecte logiciel...
print(f"{e!r}")         # __repr__ : Employe(nom='Bechir'...
print(f"{e:court}")     # __format__ : Bechir
print(f"{e:complet}")   # __format__ : Bechir (Architecte logiciel, depuis 2020)

print("\n--- Dans une liste → __repr__ ---")
equipe = [
    Employe("Bechir", "Architecte", 4500, date(2020, 3, 15)),
    Employe("Sarra", "Dev Senior", 3800, date(2021, 9, 1)),
]
print(equipe)           # affiche __repr__ de chaque élément
```

- **!r** dans les f-strings : applique repr() —met les guillemets autour des chaînes et formate les dates correctement
- **__format__** : intercepte f"{obj:spec}" —permet des formats personnalisés (court, long, JSON...)
- **règle d'or** : toujours implémenter __repr__ en premier —c'est le fallback de __str__ ET il sert dans les logs/debug
- **eval(repr(obj)) == obj** : l'idéal —si votre __repr__ produit du code Python valide qui recrée l'objet, c'est parfait

Piège classique —l'objet sans __repr__

L'idée de cet exemple est de montrer **un comportement très spécifique de Python** :

Quand un objet est **affiché à l'intérieur d'une liste**, Python utilise ******repr**** et non **str****.

C'est pour cela que l'absence de **repr** produit un affichage illisible

```
class ExempleRepr:
    def __init__(self, valeur):
        self.valeur = valeur

    #def __repr__(self):
    #    return f"AvecRepr({self.valeur!r})"

objets2 = [ExempleRepr(1), ExempleRepr(2), ExempleRepr(3)]
print(objets2)
# [AvecRepr(1), AvecRepr(2), AvecRepr(3)] ← lisible, utile en debug
```

5 □ Surcharge des opérateurs arithmétiques

Python ne limite pas +, -, *, / aux nombres —vous pouvez les redéfinir pour n'importe quelle classe qui en a un sens métier.

Table des opérateurs et leurs méthodes

Expression	Méthode appelée	Version réfléchie
a + b	a.__add__(b)	b.__radd__(a)
a - b	a.__sub__(b)	b.__rsub__(a)
a * b	a.__mul__(b)	b.__rmul__(a)
a / b	a.__truediv__(b)	b.__rtruediv__(a)
a // b	a.__floordiv__(b)	b.__rfloordiv__(a)
a % b	a.__mod__(b)	b.__rmod__(a)
a ** b	a.__pow__(b)	b.__rpow__(a)
a += b	a.__iadd__(b)	(in-place)

Exemple métier —vecteurs 2D

```
import math

class Vecteur2D:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    # --- Représentation ---
    def __repr__(self):
        return f"Vecteur2D({self.x}, {self.y})"

    def __str__(self):
        return f"({self.x}, {self.y})"

    # --- Opérateurs arithmétiques ---
    def __add__(self, other: "Vecteur2D") -> "Vecteur2D":
        return Vecteur2D(self.x + other.x, self.y + other.y)

    def __sub__(self, other: "Vecteur2D") -> "Vecteur2D":
        return Vecteur2D(self.x - other.x, self.y - other.y)
```

```

def __mul__(self, scalaire: float) -> "Vecteur2D":
    # Multiplication par un scalaire : v * 3
    return Vecteur2D(self.x * scalaire, self.y * scalaire)

def __rmul__(self, scalaire: float) -> "Vecteur2D":
    # Permet aussi : 3 * v (ordre inversé)
    return self.__mul__(scalaire)

def __neg__(self) -> "Vecteur2D":
    # Opérateur unaire -v
    return Vecteur2D(-self.x, -self.y)

def __abs__(self) -> float:
    # abs(v) → norme euclidienne
    return math.sqrt(self.x**2 + self.y**2)

def __iadd__(self, other: "Vecteur2D") -> "Vecteur2D":
    # v += autre (in-place, plus efficace)
    self.x += other.x
    self.y += other.y
    return self

# --- Comparaison ---
def __eq__(self, other) -> bool:
    return self.x == other.x and self.y == other.y

v1 = Vecteur2D(3, 4)
v2 = Vecteur2D(1, 2)

print(f"v1          = {v1}")
print(f"v2          = {v2}")
print(f"v1 + v2      = {v1 + v2}")          # (4, 6)
print(f"v1 - v2      = {v1 - v2}")          # (2, 2)
print(f"v1 * 2       = {v1 * 2}")          # (6, 8)
print(f"3 * v1       = {3 * v1}")          # (9, 12) ← __rmul__
print(f"-v1         = {-v1}")              # (-3, -4) ← __neg__
print(f"abs(v1)      = {abs(v1)}")         # 5.0 ← __abs__
print(f"v1 == v2     = {v1 == v2}")        # False

v1 += v2
print(f"v1 après += = {v1}")                # (4, 6) ← __iadd__

• __rmul__: version réfléchie — appelée quand int.__mul__(v) retourne
  NotImplemented, Python essaie alors v.__rmul__(int)
• __neg__: opérateur unaire -v — différent de __sub__ qui est binaire v1

```

- v2
- `__abs__` : intercepte `abs(obj)` —ici la norme d'un vecteur, très naturel mathématiquement
- `__iadd__` : version *in-place* de `+=` —modifie `self` directement et le retourne (plus performant que créer un nouvel objet)

6 □ Comparaisons —`__eq__`, `__lt__` et `@total_ordering`

Concept

Python dispose de 6 opérateurs de comparaison, chacun avec sa méthode :

Opérateur	Méthode	Signification
<code>==</code>	<code>__eq__</code>	égalité
<code>!=</code>	<code>__ne__</code>	différence (auto-déduit depuis <code>__eq__</code>)
<code><</code>	<code>__lt__</code>	inférieur strict
<code><=</code>	<code>__le__</code>	inférieur ou égal
<code>></code>	<code>__gt__</code>	supérieur strict
<code>>=</code>	<code>__ge__</code>	supérieur ou égal

Bonne nouvelle : avec `@total_ordering` de `functools`, implémenter `__eq__` + **une seule** des autres suffit —Python génère les 4 restantes automatiquement.

Exemple complet —classe `Version` triable

```
from functools import total_ordering

@total_ordering
class Version:
    """Représente une version sémantique : majeur.mineur.patch"""

    def __init__(self, version: str):
        parties = version.split(".")
        self.majeur = int(parties[0])
        self.mineur = int(parties[1]) if len(parties) > 1 else 0
        self.patch = int(parties[2]) if len(parties) > 2 else 0

    def _tuple(self):
```

```

        return (self.majeur, self.mineur, self.patch)

    def __eq__(self, other) -> bool:
        if not isinstance(other, Version):
            return NotImplemented # ← signal Python : "essaie l'autre sens"
        return self._tuple() == other._tuple()

    def __lt__(self, other) -> bool:
        if not isinstance(other, Version):
            return NotImplemented
        return self._tuple() < other._tuple()

# @total_ordering génère automatiquement __le__, __gt__, __ge__ depuis __eq__ + __lt__

    def __hash__(self):
        # Obligatoire si __eq__ est défini : sinon l'objet devient non-hashable
        return hash(self._tuple())

    def __str__(self):
        return f"{self.majeur}.{self.mineur}.{self.patch}"

    def __repr__(self):
        return f"Version({str(self)!r})"

v1 = Version("1.2.3")
v2 = Version("1.2.10")
v3 = Version("2.0.0")
v4 = Version("1.2.3")

print(f"v1 = {v1}, v2 = {v2}, v3 = {v3}")
print(f"v1 == v4 : {v1 == v4}") # True
print(f"v1 < v2 : {v1 < v2}") # True (3 < 10)
print(f"v1 <= v2 : {v1 <= v2}") # True ← généré par @total_ordering
print(f"v3 > v2 : {v3 > v2}") # True ← généré par @total_ordering
print(f"v2 >= v1 : {v2 >= v1}") # True ← généré par @total_ordering

# Tri d'une liste de versions — fonctionne grâce aux comparaisons
versions = [Version("3.1.0"), Version("1.0.0"), Version("2.5.3"), Version("1.0.1")]
print(f"\nTriées : {sorted(versions)}")
# [Version('1.0.0'), Version('1.0.1'), Version('2.5.3'), Version('3.1.0')]

# min / max fonctionnent aussi
print(f"Dernière version : {max(versions)}") # 3.1.0

# Utilisable dans un set ou comme clé de dict (grâce à __hash__)
versions_uniques = {Version("1.0.0"), Version("1.0.0"), Version("2.0.0")}

```

```
print(f"Set unique : {versions_uniques}") # 2 éléments seulement
```

- **NotImplemented** dans `__eq__` et `__lt__` : permet à Python d'inverser la comparaison si l'autre objet sait la gérer
- **@total_ordering** : décorateur `functools` —réduit le boilerplate de 6 méthodes à 2 (obligation : `__eq__` + un des 4 autres)
- **__hash__** : **obligatoire** si vous définissez `__eq__` —Python désactive le hash par défaut dès que vous redéfinissez l'égalité (objets mutables non-hashables par design)
- **_tuple()** : méthode privée utilitaire —centralise la logique de comparaison, évite la duplication dans chaque méthode

7 □ Transformer une classe en conteneur

En Python, les types natifs `list`, `dict`, `set` sont des conteneurs parce qu'ils implémentent un protocole précis. Vous pouvez faire pareil avec **n'importe quelle classe**.

Les méthodes du protocole conteneur

Méthode	Déclenchée par	Retour attendu
<code>__len__</code>	<code>len(obj)</code>	<code>int >= 0</code>
<code>__getitem__</code>	<code>obj[key]</code>	la valeur
<code>__setitem__</code>	<code>obj[key] = val</code>	<code>None</code>
<code>__delitem__</code>	<code>del obj[key]</code>	<code>None</code>
<code>__contains__</code>	<code>x in obj</code>	<code>bool</code>
<code>__iter__</code>	<code>for x in obj</code>	un itérateur
<code>__reversed__</code>	<code>reversed(obj)</code>	un itérateur inversé

Exemple complet d'un registre ordonné typé

```
from typing import TypeVar, Generic, Iterator, Optional
```

```
T = TypeVar("T")
```

```
class Registre(Generic[T]):
```

```
    """
```

```
    Conteneur ordonné avec accès par index ET par nom.
```

```
    Exemple : registre d'employés, catalogue de produits.
```

```
    """
```

```

def __init__(self, nom: str):
    self.nom = nom
    self._items: list = []
    self._index: dict = {} # nom → position

def ajouter(self, cle: str, valeur: T):
    if cle in self._index:
        raise KeyError(f"Clé '{cle}' déjà présente")
    self._index[cle] = len(self._items)
    self._items.append((cle, valeur))

# --- Protocole conteneur ---

def __len__(self) -> int:
    return len(self._items)

def __getitem__(self, key):
    if isinstance(key, int):
        return self._items[key][1] # accès par index numérique
    elif isinstance(key, str):
        idx = self._index[key]
        return self._items[idx][1] # accès par nom
    elif isinstance(key, slice):
        return [v for _, v in self._items[key]] # slicing
    raise TypeError(f"Index invalide : {type(key)}")

def __setitem__(self, cle: str, valeur: T):
    if cle in self._index:
        idx = self._index[cle]
        self._items[idx] = (cle, valeur) # mise à jour
    else:
        self.ajouter(cle, valeur) # ajout

def __delitem__(self, cle: str):
    if cle not in self._index:
        raise KeyError(cle)
    idx = self._index.pop(cle)
    self._items.pop(idx)
    # Recalculer les indices après suppression
    self._index = {k: i for i, (k, _) in enumerate(self._items)}

def __contains__(self, cle: str) -> bool:
    return cle in self._index

def __iter__(self) -> Iterator[T]:

```

```

        return (valeur for _, valeur in self._items)

    def __reversed__(self) -> Iterator[T]:
        return (valeur for _, valeur in reversed(self._items))

    def __str__(self):
        contenu = ", ".join(f"{k}={v}" for k, v in self._items)
        return f"Registre({self.nom!r})[{contenu}]"

# Utilisation
reg = Registre("Employés")
reg.ajouter("bechir", "Architecte")
reg.ajouter("sarra", "Dev Senior")
reg.ajouter("omar", "QA Lead")

print(f"Taille      : {len(reg)}")           # 3
print(f"Par index   : {reg[0]}")             # Architecte
print(f"Par nom     : {reg['sarra']}")        # Dev Senior
print(f"Slice [1:]  : {reg[1:]}")            # ['Dev Senior', 'QA Lead']
print(f"'omar' in   : {'omar' in reg}")       # True
print(f"'ali' in    : {'ali' in reg}")       # False

print("\nItération :")
for employe in reg:
    print(f" - {employe}")

print("\nItération inversée :")
for employe in reversed(reg):
    print(f" - {employe}")

reg["bechir"] = "CTO"    # mise à jour
del reg["omar"]          # suppression
print(f"\nAprès modifications : {reg}")

```

- `__getitem__` avec `isinstance(key, int|str|slice)` : permet des accès flexibles —par index, par nom, par tranche— comme les DataFrame pandas
- `__iter__` retourne un **générateur** ici (expression génératrice) —plus économe en mémoire qu'une liste
- `__reversed__` : sans cette méthode, `reversed(obj)` nécessite que `__len__` et `__getitem__` soient définis —l'implémenter explicitement est plus propre
- `Generic[T]` et `TypeVar` : rendent le registre générique et compatible avec les outils de typage statique (mypy, IDEs)

8 □ Interception des attributs

Python offre trois méthodes pour intercepter les accès aux attributs —chacune avec un périmètre et des dangers différents.

Les trois niveaux d'interception

obj.x	→	<code>__getattribute__</code>	(TOUJOURS, même si x existe)
		↓ si <code>AttributeError</code>	
		<code>__getattr__</code>	(SEULEMENT si x n'existe pas)
obj.x = valeur	→	<code>__setattr__</code>	(TOUJOURS)
del obj.x	→	<code>__delattr__</code>	(TOUJOURS)

Exemple progressif —des plus sûrs aux plus dangereux

```
# — Niveau 1 : __getattr__ — Le plus courant, le plus sûr —————
class ObjetDynamique:
    """Retourne une valeur par défaut pour tout attribut manquant."""

    def __init__(self, **kwargs):
        self.__dict__.update(kwargs)

    def __getattr__(self, name: str):
        # Appelé SEULEMENT si l'attribut n'existe pas dans __dict__
        # Jamais de récursion infinie ici — contrairement à __getattribute__
        print(f" [getattr] '{name}' introuvable → valeur par défaut")
        return None

obj = ObjetDynamique(nom="Bechir", age=40)
print(obj.nom)          # Bechir          ← attribut existant, __getattr__ NON appelé
print(obj.email)        # None            ← attribut manquant, __getattr__ appelé
print(obj.telephone)    # None            ← idem

# — Niveau 2 : __setattr__ — validation à l'écriture —————
class Typé:
    """Valide le type de chaque attribut à l'assignation."""

    SCHEMA = {"nom": str, "age": int, "salaire": float}

    def __setattr__(self, name: str, value):
        if name in self.SCHEMA:
            attendu = self.SCHEMA[name]
```

```

        if not isinstance(value, attendu):
            raise TypeError(
                f"'{name}' attend {attendu.__name__}, reçu {type(value).__name__}"
            )
        # OBLIGATOIRE : déléguer à super() pour vraiment stocker l'attribut
        # Ne JAMAIS écrire self.name = value ici → récursion infinie !
        super().__setattr__(name, value)

t = Typé()
t.nom = "Bechir"      # str
t.age = 40            # int
t.salaire = 3500.0    # float

try:
    t.age = "quarante" # str au lieu de int
except TypeError as e:
    print(f"Erreur de type : {e}")

# — Niveau 3 : __getattr__ — Le plus puissant, Le plus dangereux —
class Journalise:
    """Journalise TOUS Les accès aux attributs (même ceux qui existent)."""

    def __init__(self, valeur):
        # On doit contourner notre propre __getattr__ pour éviter la récursion
        object.__setattr__(self, "valeur", valeur)
        object.__setattr__(self, "_log", [])

    def __getattr__(self, name: str):
        # Appelé pour ABSOLUMENT TOUT accès — y compris self._log et self.valeur !
        # DANGER : accéder à self.n'importe quoi ici rappelle __getattr__ → récursion
        # Solution : toujours utiliser object.__getattr__ pour les accès internes
        log = object.__getattr__(self, "_log")
        log.append(name)
        return object.__getattr__(self, name) # délégation obligatoire

j = Journalise(42)
_ = j.valeur      # accès
_ = j.valeur      # accès
_ = j._log        # accès

print(f"Attributs accédés : {object.__getattr__(j, '_log')}")
# ['valeur', 'valeur', '_log']

• __getattr__ : le piège n'existe pas ici —il n'est appelé que si l'attribut
est absent → usage courant et sûr

```

- `__setattr__` : le piège classique est `self.x = val` à l'intérieur → rappelle `__setattr__` → récursion infinie → **toujours** utiliser `super().__setattr__(self, ...)`
 - `__getattr__` : intercepte **tout**, y compris les accès internes (`self.x`) → **toujours** déléguer via `object.__getattr__(self, name)` pour les accès internes
 - Règle de priorité : préférer `__getattr__` (sûr) → puis `__setattr__` (avec `super()`) → `__getattr__` en dernier recours seulement
-

9 □ Objet callable — `__call__`

`__call__` transforme un objet en **callable** —il peut être appelé comme une fonction avec `obj(...)`. Contrairement à une simple fonction, l'objet **conserve un état** entre les appels.

Cas d'usage 1 : Décorateur avec état (compteur d'appels)

```
class Compte:
    """Compte le nombre de fois qu'une fonction est appelée."""

    def __init__(self, fonction):
        self.fonction = fonction
        self.appels = 0

    def __call__(self, *args, **kwargs):
        self.appels += 1
        print(f" [Appel #{self.appels}] {self.fonction.__name__}()")
        return self.fonction(*args, **kwargs)

    def reset(self):
        self.appels = 0

@Compte
def calculer(x, y):
    return x + y

print(calculer(1, 2))    # [Appel #1] calculer() → 3
print(calculer(3, 4))    # [Appel #2] calculer() → 7
print(calculer(5, 6))    # [Appel #3] calculer() → 11
print(f"Total appels : {calculer.appels}")    # 3
```

Cas d'usage 2 : Pipeline de transformations configurable

```
class Pipeline:
    """Enchaîne des fonctions de transformation."""
```

```

def __init__(self, *etapes):
    self.etapes = etapes

def __call__(self, valeur):
    for etape in self.etapes:
        valeur = etape(valeur)
    return valeur

def __repr__(self):
    noms = " → ".join(f.__name__ for f in self.etapes)
    return f"Pipeline({noms})"

# Définition des étapes
def supprimer_espaces(s): return s.strip()
def mettre_en_minuscules(s): return s.lower()
def remplacer_espaces(s): return s.replace(" ", "_")

nettoyer = Pipeline(supprimer_espaces, mettre_en_minuscules, remplacer_espaces)

print(f"\nPipeline : {nettoyer}")
entrees = [" Bonjour Monde ", " PYTHON EST PUISSANT ", " Hello World "]
for texte in entrees:
    print(f" {texte!r:30} → {nettoyer(texte)!r}")

[Appel #1] calculer()
3
[Appel #2] calculer()
7
[Appel #3] calculer()
11
Total appels : 3

Pipeline : supprimer_espaces → mettre_en_minuscules → remplacer_espaces
' Bonjour Monde ' → 'bonjour_monde'
' PYTHON EST PUISSANT ' → 'python_est_puissant'
' Hello World ' → 'hello_world'

• @Compte : la classe elle-même est utilisée comme décorateur —calculer
devient une instance de Compte qui est callable
• *args, **kwargs dans __call__ : transmet tous les arguments à la
fonction originale —rend le décorateur transparent
• avantage vs fonction : le pipeline “se souvient” de ses étapes —on peut
l’inspecter (print(nettoyer)), le sérialiser, le passer comme argument
• callable(obj) : retourne True si __call__ est défini —utile pour véri-
fier avant d’appeler dynamiquement

```

□ Gestionnaire de contexte — `__enter__` / `__exit__`

Le protocole `with` garantit qu'une ressource est **toujours libérée**, même si une exception se produit. C'est le mécanisme le plus robuste de Python pour la gestion de ressources.

```
with obj as x:
    # bloc
```

↓ équivaut à :

```
x = obj.__enter__()
try:
    # bloc
except:
    obj.__exit__(exc_type, exc_val, exc_tb)
    raise
else:
    obj.__exit__(None, None, None)
```

Exemple simple

```
class FichierTexte:
```

```
    def __init__(self, nom_fichier):
        self.nom_fichier = nom_fichier
        self.fichier = None

    def __enter__(self):
        print("Ouverture du fichier")
        self.fichier = open(self.nom_fichier, "w")
        return self.fichier  # valeur associée à "as f"

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Fermeture du fichier")
        self.fichier.close()
```

```
with FichierTexte("demo.txt") as f:
    f.write("Bonjour\n")
    f.write("Python\n")**`__exit__` retourne `False` (ou `None`)** : l'exception se pr
```

Pourquoi c'est utile:

```
f = open("demo.txt", "w")
try:
    f.write("Bonjour")
finally:
    f.close()
```

Ce que fait python derrière les scenes:

```
obj = FichierTexte("demo.txt")

f = obj.__enter__()

try:
    f.write("Bonjour")

except Exception as e:
    obj.__exit__(type(e), e, e.__traceback__)
    raise

else:
    obj.__exit__(None, None, None)
```

- **__exit__** retourne **True** : l'exception est **supprimée** —usage rare, réservé aux rollbacks contrôlés
- **exc_type, exc_val, exc_tb**: None, None, None si aucune exception —tester if **exc_type is None** pour distinguer les deux cas
- **alternative légère** : @contextmanager de contextlib —pour les cas simples sans état complexe, un générateur avec yield suffit

□ Récapitulatif —les méthodes magiques les plus importantes à maîtriser

```
__init__ / __new__ · __str__ / __repr__ / __format__ ·
__add__ / __radd__ / __iadd__ · __eq__ / __hash__ +
@total_ordering · __len__ / __getitem__ / __iter__ ·
__getattr__ / __setattr__ (avec super()) · __call__ ·
__enter__ / __exit__
```

□ 20 Questions intelligentes –Niveau intermédiaire structuré

1) Pourquoi eq doit retourner NotImplemented ?

Énoncé

Que se passe-t-il si on compare avec un type différent ?

Solution

```
"""
Explication
- Si le type est incompatible, retourner NotImplemented
- Permet à Python d'essayer l'opération inverse
"""

class A:
    def __eq__(self, other):
        if not isinstance(other, A):
            return NotImplemented
        return True

print(A() == 5)
```

2) Pourquoi redéfinir eq impose souvent de redéfinir hash ?

```
"""
Explication
- Si __eq__ change la logique d'égalité
- __hash__ doit rester cohérent
- Sinon dictionnaires et sets se comportent mal
"""
```

```
class Point:

    def __init__(self, x):
        self.x = x

    def __eq__(self, other):
        return self.x == other.x

    def __hash__(self):
        return hash(self.x)
```

```
p1 = Point(1)
p2 = Point(1)

print(set([p1, p2]))
```

3) Pourquoi `lt` seul ne suffit pas pour toutes les comparaisons ?

Énoncé

Si on implémente seulement `lt`, pourquoi `<=` ou `>` peuvent échouer ?

Solution

Créer le fichier `q3_total_ordering.py` :

```
"""
Explication
- Implémenter seulement __lt__ et __eq__ ne fournit pas automatiquement
  toutes les autres comparaisons.
- functools.total_ordering génère les méthodes manquantes.
"""
```

```
from functools import total_ordering

@total_ordering
class Produit:

    def __init__(self, prix):
        self.prix = prix

    def __eq__(self, other):
        return self.prix == other.prix

    def __lt__(self, other):
        return self.prix < other.prix

p1 = Produit(10)
p2 = Produit(20)

print("p1 <= p2 :", p1 <= p2)
print("p1 > p2 :", p1 > p2)
```

4) Que se passe-t-il si add retourne un mauvais type ?

Énoncé

Pourquoi retourner un type inattendu casse la cohérence métier ?

Solution

Créer le fichier q4_add_bug.py :

```
"""
Explication
- Une surcharge d'opérateur doit respecter le modèle métier.
- Retourner un type incohérent rend l'objet inutilisable ensuite.
"""

class Compte:

    def __init__(self, solde):
        self.solde = solde

    def __add__(self, other):
        return self.solde + other.solde # mauvais retour (int)

c1 = Compte(100)
c2 = Compte(200)

resultat = c1 + c2
print("Type du résultat :", type(resultat))
```

5) Pourquoi len doit retourner un entier ?

Énoncé

Que se passe-t-il si **len** retourne autre chose qu'un int ?

Solution

Créer le fichier q5_len_error.py :

```
"""
Explication
- len(obj) exige un entier >= 0.
```

```
- Retourner autre chose provoque TypeError.  
"""
```

```
class MauvaiseListe:  
  
    def __len__(self):  
        return "3" # erreur volontaire  
  
try:  
    print(len(MauvaiseListe()))  
except TypeError as e:  
    print("Erreur :", e)
```

6) Différence entre getattr et getattribute ?

Énoncé

Pourquoi **getattribute** est plus dangereux ?

Solution

Créer le fichier q6_getattribute.py :

```
"""  
Explication  
- __getattribute__ intercepte TOUS les accès.  
- Il faut utiliser super().__getattribute__ sinon récursion infinie.  
"""
```

```
class Demo:  
  
    def __getattribute__(self, name):  
        print("Accès intercepté :", name)  
        return super().__getattribute__(name)  
  
    def hello(self):  
        return "Bonjour"  
  
d = Demo()  
print(d.hello())
```

7) Pourquoi setattr peut provoquer une récursion infinie ?

Créer le fichier q7_setattr.py :

```
"""
Explication
- Affecter self.attribut dans __setattr__
  rappelle __setattr__ → boucle infinie.
- Il faut utiliser super().__setattr__.
"""
```

```
class Demo:
```

```
    def __setattr__(self, name, value):
        print("Attribut modifié :", name)
        super().__setattr__(name, value)
```

```
d = Demo()
d.x = 10
print("d.x =", d.x)
```

8) Pourquoi iter doit retourner un itérateur ?

Créer le fichier q8_iter.py :

```
"""
Explication
- __iter__ doit retourner un objet implémentant __next__.
- Retourner une liste est acceptable car elle est itérable.
"""
```

```
class Demo:
```

```
    def __init__(self):
        self.data = [1, 2, 3]

    def __iter__(self):
        return iter(self.data)
```

```
for x in Demo():
    print("Valeur :", x)
```

9) Pourquoi call peut remplacer une fonction ?

Créer le fichier q9_call.py :

```
"""
Explication
- Un objet avec __call__ peut encapsuler un comportement configurable.
"""
```

```
class Additionneur:

    def __init__(self, increment):
        self.increment = increment

    def __call__(self, valeur):
        return valeur + self.increment

add5 = Additionneur(5)
print("Résultat :", add5(10))
```

10) Que se passe-t-il si enter ne retourne rien ?

Créer le fichier q10_context.py :

```
"""
Explication
- La valeur retournée par __enter__ est assignée après 'as'.
- Si None → variable vaut None.
"""
```

```
class Demo:

    def __enter__(self):
        print("Entrée")
        # pas de return

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Sortie")

with Demo() as d:
    print("Valeur de d :", d)
```

11) Pourquoi exit reçoit trois paramètres ?

Créer le fichier q11_exit.py :

```
"""
Explication
- exc_type, exc_val, exc_tb permettent de gérer les exceptions.
"""

class Demo:

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        print("Exception capturée :", exc_type)

with Demo():
    raise ValueError("Erreur volontaire")
```

12) Pourquoi contains est plus rapide que itération manuelle ?

Créer le fichier q12_contains.py :

```
"""
Explication
- __contains__ est utilisé pour l'opérateur 'in'.
"""

class Demo:

    def __init__(self):
        self.data = [1, 2, 3]

    def __contains__(self, item):
        return item in self.data

d = Demo()
print("2 in d :", 2 in d)
```

13) Pourquoi bool est utile ?

Créer le fichier q13_bool.py :

```
"""
Explication
- __bool__ définit la valeur de vérité d'un objet.
"""
```

```
class Demo:
```

```
    def __bool__(self):
        return False
```

```
d = Demo()
```

```
if d:
    print("Vrai")
else:
    print("Faux")
```

14) Pourquoi hash doit être cohérent avec eq ?

(complément démonstratif)

```
"""
Explication
- Deux objets égaux doivent avoir le même hash.
"""
```

```
class Point:
```

```
    def __init__(self, x):
        self.x = x

    def __eq__(self, other):
        return self.x == other.x

    def __hash__(self):
        return hash(self.x)
```

```
p1 = Point(1)
p2 = Point(1)
```

```
print("p1 == p2 :", p1 == p2)
print("Hash identique :", hash(p1) == hash(p2))
```

15) Pourquoi slots empêche les attributs dynamiques ?

```
"""
Explication
- __slots__ limite les attributs possibles.
"""
```

```
class Point:

    __slots__ = ("x", "y")

    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
p = Point(1, 2)

try:
    p.z = 3
except AttributeError as e:
    print("Erreur :", e)
```

16) Que se passe-t-il si getitem lève une mauvaise exception ?

```
"""
Explication
- IndexError est attendu pour index invalide.
"""
```

```
class Demo:

    def __getitem__(self, index):
        raise ValueError("Mauvaise exception")

try:
```

```
d = Demo()
d[0]
except Exception as e:
    print("Type erreur :", type(e))
```

17) Pourquoi del est dangereux ?

```
"""
Explication
- __del__ est appelé lors du garbage collection.
- Moment non déterministe.
"""
```

```
class Demo:

    def __del__(self):
        print("Objet détruit")
```

```
d = Demo()
del d
```

18) Différence entre immuable et mutable dans new ?

```
"""
Explication
- Pour les types immuables (comme int), __new__ est essentiel.
"""
```

```
class MyInt(int):

    def __new__(cls, value):
        print("Création immuable")
        return super().__new__(cls, value)
```

```
x = MyInt(10)
print(x)
```

19) Pourquoi retourner NotImplemented est important en arithmétique ?

```
"""
Explication
- Permet à Python d'essayer l'opération inverse.
"""
```

```
class A:
    def __add__(self, other):
        return NotImplemented

class B:
    def __radd__(self, other):
        return "Opération inversée"

a = A()
b = B()

print(a + b)
```

20) Pourquoi le MRO est crucial avec les méthodes magiques ?

```
"""
Explication
- En héritage multiple, l'ordre MRO détermine quelle méthode magique est appelée.
"""
```

```
class A:
    def __str__(self):
        return "A"

class B:
    def __str__(self):
        return "B"

class C(A, B):
    pass

c = C()
```

```
print("Affichage :", c)  
print("MRO :", [cls.__name__ for cls in C.mro()])
```